```
var seed = HKDF.expand(macKey, "nacl-signing-key-seed",    #A
        32, "NaCl");    #A
var keyPair =
Crypto.seedSigningKeyPair(seed.getEncoded());    #B
```

#A Use HKDF to generate a seed
#B Derive a signing keypair from the seed

> **CAUTION** The algorithms used by Salty Coffee, X25519 and Ed25519, are designed to safely allow this. The same is not true of other algorithms.

Although generating a handful of keys from a master key may not seem like much of a saving, the real value comes from the ability to generate keys programmatically that are the same on all servers. For example, you can include the current date in the context string and automatically derive a fresh encryption key each day without needing to distribute a new key to every server. If you include the context string in the encrypted data, for example as the `kid` header in an encrypted JWT, then you can quickly re-derive the same key whenever you need without storing previous keys.

Facebook CATs

As you might expect, Facebook needs to run many services in production with very many clients connecting to each service. At the huge scale they are running at, public key cryptography is deemed too expensive, but they still want to use strong authentication between clients and services. Every request and response between a client and a service is authenticated with HMAC using a key that is unique to that client-service pair. These signed HMAC tokens are known as *Crypto Auth Tokens*, or CATs, and are a bit like signed JWTs. To avoid storing, distributing, and managing thousands of keys, Facebook uses key derivation heavily. A central key distribution service stores a master key. Clients and services authenticate to the key distribution service to get keys based on their identity. The key for a service with the name “AuthService” is calculated using `KDF(masterKey,`